Part 1: Small Web Service

The project uses the Node.js HTTP service from the starter GitHub repository. Environment variables configure the listening port, service name, and message. The request counter increments with each successful `GET /` request but not health checks. Since the counter is stored in process memory, it resets when the application restarts and is not shared between replicas.

A Dockerfile packages the application as `cs554-midterm-app:1.0`. The same image is used for Docker Compose inside the Ubuntu VirtualBox VM and Kubernetes in the local k3d cluster running through WSL. The image was loaded into the k3d node's container image store by streaming `docker save` into `ctr images import`. The application writes startup and HTTP request messages to standard output, making them available through container logs.

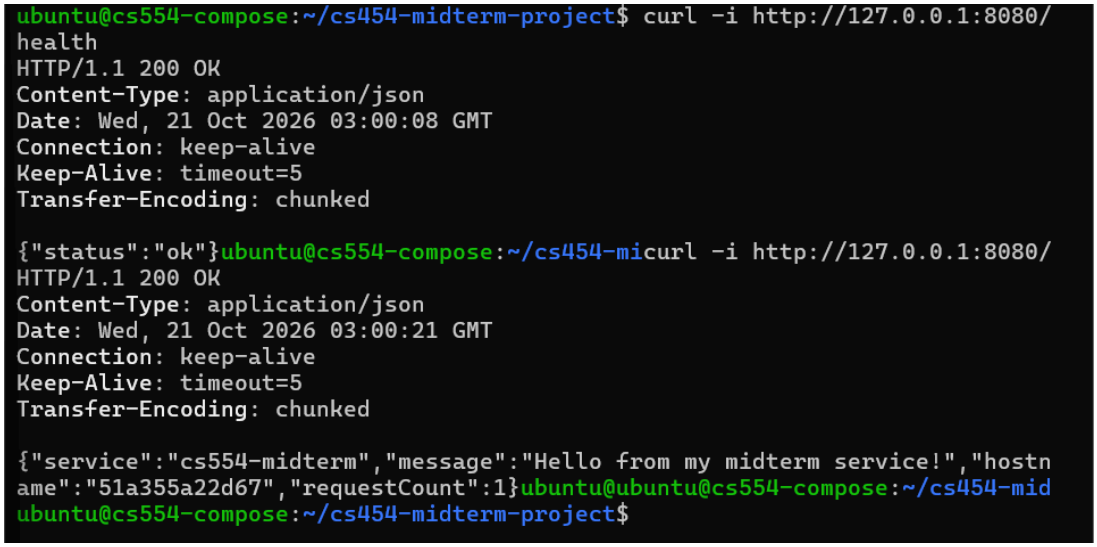


Figure 1. Successful responses from `/health` and `/` inside the Ubuntu VM.

Part 2: Docker Compose Inside a Linux VM

The application image was built in Ubuntu WSL and transferred into the VM using `docker save` and `docker load`. The service was started with `docker compose up -d --wait`.

Docker Compose runs the application inside an Ubuntu VirtualBox VM. The Compose configuration supplies the environment variables and maps VM port 8080 to container port 8080. VirtualBox NAT forwards Windows host port 18081 to VM port 8080. Requests from Windows to both endpoints returned HTTP 200, confirming successful host-to-VM connectivity.

```
PS C:\Users\sarah> curl.exe -i http://127.0.0.1:18081/health
HTTP/1.1 200 OK
Content-Type: application/json
Date: Wed, 21 Oct 2026 03:33:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"status":"ok"}
PS C:\Users\sarah> curl.exe -i http://127.0.0.1:18081/
HTTP/1.1 200 OK
Content-Type: application/json
Date: Wed, 21 Oct 2026 03:34:06 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"service":"cs554-midterm","message":"Hello from my midterm service!","hostn
ame":"51a355a22d67","requestCount":2}
PS C:\Users\sarah>
```

Figure 2. Successful requests from Windows to the application through NAT-forwarded port 18081.

The request travels from the Windows host through VirtualBox NAT to the Ubuntu VM, where Docker forwards it to the Node.js process inside the container. The VM runs its own operating system, while the container shares the VM's Linux kernel.

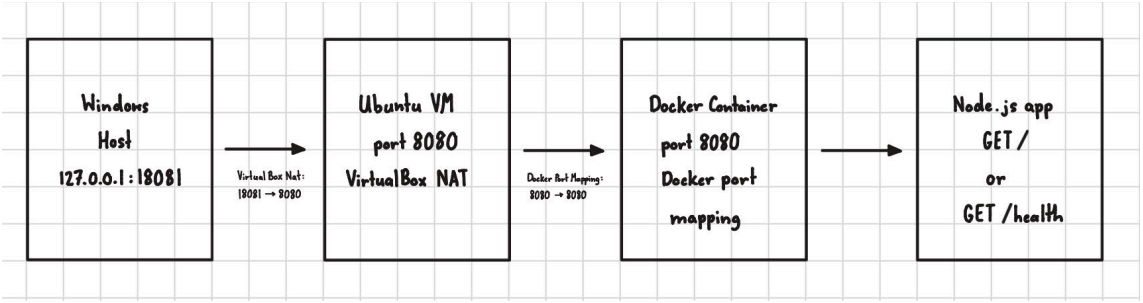


Figure 3. Host-to-VM request path for the Docker Compose deployment.

Docker Compose reported the container as healthy with port 8080 published. Container logs recorded `GET /health` requests with HTTP 200, confirming that application request information was written to standard output.

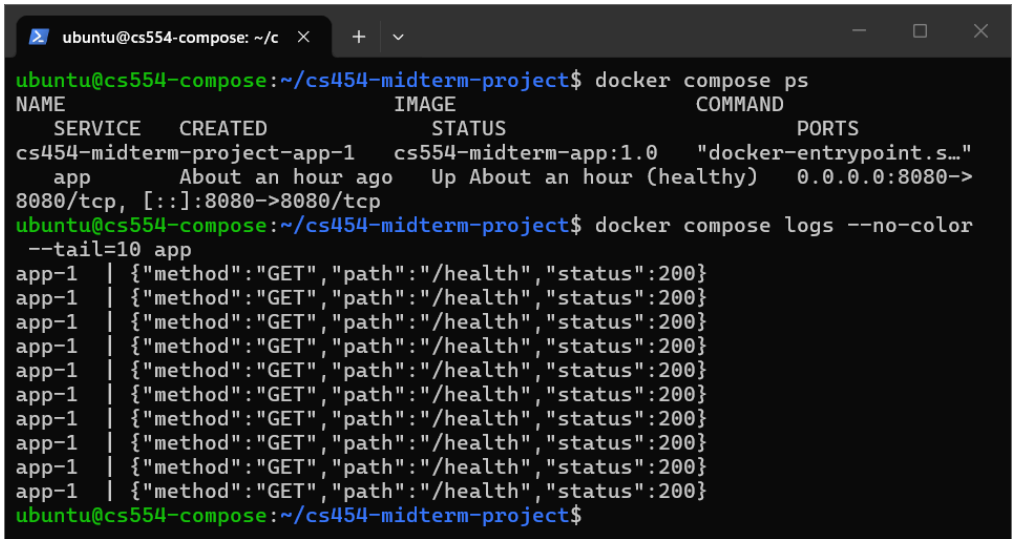


Figure 4. Docker Compose status and application logs showing successful health requests.

Two consecutive requests increased the counter from 3 to 4. After restarting the Compose service, the first request returned a count of 1 while the container hostname remained unchanged. The counter reset because it is stored in application memory rather than persistent storage.

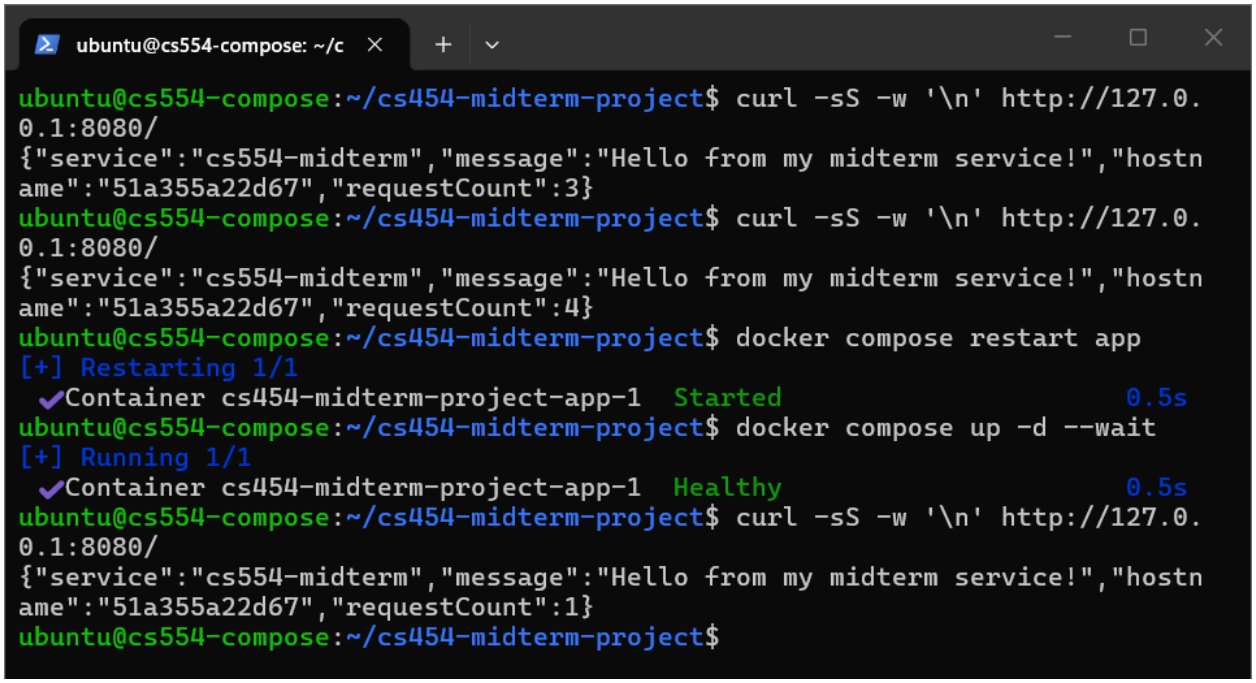


Figure 5. Request counter before and after restarting the Compose service.

Part 3: Kubernetes Deployment

The Kubernetes deployment runs in a local k3d cluster inside Ubuntu WSL, separate from the VirtualBox VM used for Compose. The Deployment specifies two replicas using `cs554-midterm-app:1.0`. Both application Pods were Running and Ready.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get deployment,pods,service -o wide
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES                  SELECTOR
deployment.apps/cs554-midterm   2/2     2            2           11h   api          cs554-midterm-app:1.0   app=cs554-midterm

NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE                        NOMINATED NODE   READINESS GATES
pod/cs554-midterm-5f85d994bf-c86nn   1/1     Running   0          86m   10.42.0.34   k3d-cs554-midterm-server-0   <none>           <none>
pod/cs554-midterm-5f85d994bf-vnc72   1/1     Running   0          86m   10.42.0.36   k3d-cs554-midterm-server-0   <none>           <none>

NAME                    TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE   SELECTOR
service/cs554-midterm   ClusterIP   10.43.18.185   <none>        80/TCP    11h   app=cs554-midterm
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 6. Kubernetes Deployment with two Ready application replicas and a ClusterIP Service.

A ClusterIP Service selects Pods labeled `app=cs554-midterm`. The in-cluster client resolves the Service name `cs554-midterm` through cluster DNS to the Service's ClusterIP address. Kubernetes routes requests sent to Service port 80 to an eligible Ready Pod on application port 8080. The Service selector determines which Pods belong to the Service. A request may reach either one of the pods.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm exec midterm-client -- wget -S -O - http://cs554-midterm/health
--2026-10-21 04:28:39--  http://cs554-midterm/health
Resolving cs554-midterm (cs554-midterm)... 10.43.18.185
Connecting to cs554-midterm (cs554-midterm)|10.43.18.185|:80... connected.
HTTP request sent, awaiting response... {"status":"ok"}
  HTTP/1.1 200 OK
  Content-Type: application/json
  Date: Wed, 21 Oct 2026 04:28:39 GMT
  Connection: keep-alive
  Keep-Alive: timeout=5
  Transfer-Encoding: chunked
Length: unspecified [application/json]
Saving to: 'STDOUT'

     0K                                                       1.90M=0s

2026-10-21 04:28:39 (1.90 MB/s) - written to stdout [15]

root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm exec midterm-client -- wget -S -O - http://cs554-midterm/
--2026-10-21 04:28:47--  http://cs554-midterm/
Resolving cs554-midterm (cs554-midterm)... 10.43.18.185
Connecting to cs554-midterm (cs554-midterm)|10.43.18.185|:80... connected.
HTTP request sent, awaiting response...
  HTTP/1.1 200 OK
  Content-Type: application/json
  Date: Wed, 21 Oct 2026 04:28:47 GMT
  Connection: keep-alive
  Keep-Alive: timeout=5
  Transfer-Encoding: chunked
Length: unspecified [application/json]
Saving to: 'STDOUT'

     0K                                                       20.2M=0s

2026-10-21 04:28:47 (20.2 MB/s) - written to stdout [131]

{"service":"cs554-midterm","message":"Hello from my midterm service!","hostname":"cs554-midterm-5f85d994bf-c86nn","requestCount":1}root@LAPTOP-B3C65M98:~/cs
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 7. Successful requests through the Kubernetes Service.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get deployment cs554-midterm
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
cs554-midterm   2/2     2            2           11h
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pods -l app=cs554-midterm -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE                        NOMINATED NODE   READINESS GATES
cs554-midterm-5f85d994bf-c86nn   1/1     Running   0          96m   10.42.0.34   k3d-cs554-midterm-server-0   <none>           <none>
cs554-midterm-5f85d994bf-vnc72   1/1     Running   0          96m   10.42.0.36   k3d-cs554-midterm-server-0   <none>           <none>
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 8. Two Ready application Pods before deleting one replica.

After application Pod `cs554-midterm-5f85d994bf-c86nn` was deleted, Kubernetes created `cs554-midterm-5f85d994bf-qpxmv` and restored two Ready replicas. The Deployment's ReplicaSet created the replacement to maintain the desired replica count of two. Unlike the Compose restart, which reused the existing container, Kubernetes created a new Pod with a different name.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get deployment cs554-midterm
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
cs554-midterm   2/2     2            2           11h
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pods -l app=cs554-midterm -o wide
NAME                            READY   STATUS    RESTARTS   AGE   IP           NODE                          NOMINATED NODE   READINESS GATES
cs554-midterm-5f85d994bf-c86nn   1/1     Running   0          96m   10.42.0.34   k3d-cs554-midterm-server-0   <none>           <none>
cs554-midterm-5f85d994bf-vnc72   1/1     Running   0          96m   10.42.0.36   k3d-cs554-midterm-server-0   <none>           <none>
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm delete pod cs554-midterm-5f85d994bf-c86nn
pod "cs554-midterm-5f85d994bf-c86nn" deleted from cs554-midterm namespace
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm rollout status deployment/cs554-midterm --timeout=120s
deployment "cs554-midterm" successfully rolled out
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get deployment cs554-midterm
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
cs554-midterm   2/2     2            2           11h
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pods -l app=cs554-midterm -o wide
NAME                            READY   STATUS    RESTARTS   AGE   IP           NODE                          NOMINATED NODE   READINESS GATES
cs554-midterm-5f85d994bf-qpxmv   1/1     Running   0          21s   10.42.0.38   k3d-cs554-midterm-server-0   <none>           <none>
cs554-midterm-5f85d994bf-vnc72   1/1     Running   0          98m   10.42.0.36   k3d-cs554-midterm-server-0   <none>           <none>
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 9. Kubernetes replaces a deleted application Pod and restores two Ready replicas.

Each application container requests 100m CPU and 64Mi of memory, with limits of 500m CPU and 128Mi. These provide modest starting allocations for the small HTTP service. Requests are used when scheduling Pods and do not represent fixed resource usage. CPU limits may cause throttling, while exceeding the memory limit can result in container termination.

The readiness probe checks `/health` to determine whether a Pod can receive Service traffic. A failed readiness probe removes the Pod from eligible Service endpoints without automatically restarting its container. This behavior is examined in Part 4.

```
Pod Template:
  Labels:  app=cs554-midterm
  Containers:
   api:
    Image:      cs554-midterm-app:1.0
    Port:       8080/TCP (http)
    Host Port:  0/TCP (http)
    Limits:
      cpu:     500m
      memory:  128Mi
    Requests:
      cpu:      100m
      memory:   64Mi
    Readiness:  exec [wget -q -O /dev/null http://127.0.0.1:8080/health] delay=2s timeout=1s period=2s #success=1 #failure=2
    Environment:
      PORT:              8080
      SERVICE_NAME:      cs554-midterm
      MESSAGE:           Hello from my midterm service!
      NOT_READY_FILE:    /tmp/midterm-not-ready
    Mounts:              <none>
  Volumes:               <none>
  Node-Selectors:        <none>
  Tolerations:           <none>
```

Figure 10. Application container resource requests, limits, and readiness configuration.

Part 4: Controlled Troubleshooting

The working baseline had two Ready application Pods labeled `app=cs554-midterm`. The Service used the same selector, and its EndpointSlice listed both Pods as ready. A request from the in-cluster client through the Service returned HTTP 200.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pods -l app=cs554-midterm --show-labels
NAME                              READY   STATUS    RESTARTS   AGE    LABELS
cs554-midterm-5f85d994bf-qpxmv   1/1     Running   0          19m    app=cs554-midterm,pod-template-hash=5f85d994bf
cs554-midterm-5f85d994bf-vnc72   1/1     Running   0          117m   app=cs554-midterm,pod-template-hash=5f85d994bf
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get service cs554-midterm -o wide
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE   SELECTOR
cs554-midterm   ClusterIP   10.43.18.185   <none>        80/TCP    11h   app=cs554-midterm
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get endpointslices -l kubernetes.io/service-name=cs554-midterm -o yaml
apiVersion: v1
items:
- addressType: IPv4
  apiVersion: discovery.k8s.io/v1
  endpoints:
  - addresses:
    - 10.42.0.36
    conditions:
      ready: true
      serving: true
      terminating: false
    nodeName: k3d-cs554-midterm-server-0
    targetRef:
      kind: Pod
      name: cs554-midterm-5f85d994bf-vnc72
      namespace: cs554-midterm
      uid: 83706dc2-f5f7-4ded-a664-e3b598dd830d
  - addresses:
    - 10.42.0.38
    conditions:
      ready: true
      serving: true
      terminating: false
    nodeName: k3d-cs554-midterm-server-0
    targetRef:
      kind: Pod
      name: cs554-midterm-5f85d994bf-qpxmv
      namespace: cs554-midterm
      uid: ddb2998f-d761-4363-8cd5-25e3ab231c9e
  kind: EndpointSlice
  metadata:
    annotations:
      endpoints.kubernetes.io/last-change-trigger-time: "2026-10-21T04:33:24Z"
```

Figure 11a. Matching Pod labels and Service selector, with two ready endpoints.

```
      kubernetes.io/service-name: cs554-midterm
    name: cs554-midterm-lx8mq
    namespace: cs554-midterm
    ownerReferences:
    - apiVersion: v1
      blockOwnerDeletion: true
      controller: true
      kind: Service
      name: cs554-midterm
      uid: 37566bdf-b083-45b2-b440-f42a5c5417aa
    resourceVersion: "9689"
    uid: ca17d845-e2e9-45fc-8028-e960bff59597
  ports:
  - name: http
    port: 8080
    protocol: TCP
kind: List
metadata:
  resourceVersion: ""
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm exec midterm-client -- wget -S -T 3 -t 1 -O - http://cs554-midterm/
--2026-10-21 04:53:08--  http://cs554-midterm/
Resolving cs554-midterm (cs554-midterm)... 10.43.18.185
Connecting to cs554-midterm (cs554-midterm)|10.43.18.185|:80... connected.
HTTP request sent, awaiting response...
  HTTP/1.1 200 OK
  Content-Type: application/json
  Date: Wed, 21 Oct 2026 04:53:08 GMT
  Connection: keep-alive
  Keep-Alive: timeout=5
  Transfer-Encoding: chunked
Length: unspecified [application/json]
Saving to: 'STDOUT'

     0K                                                       {"service":"cs554-midterm","message":"Hello from my midterm service!","hostname":"cs554-midter
m-5f85d994bf-vnc72","requestCount":1} 19.8M=0s

2026-10-21 04:53:08 (19.8 MB/s) - written to stdout [131]

root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 11b. Successful Service request before introducing the selector mismatch.

The Service selector was changed from `app=cs554-midterm` to `app=cs554-no-match`. Both application Pods remained Running and Ready with their original labels, but the Service had no matching endpoints. A request through the Service failed with “Connection refused” and exit code 4. This showed that healthy Pods alone do not guarantee Service connectivity.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm patch service cs554-midterm --type=merge -p '{"spec":{"selector":{"app":"cs554-no-mat
ch"}}}'
service/cs554-midterm patched
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pods -l app=cs554-midterm --show-labels
NAME                              READY   STATUS    RESTARTS   AGE    LABELS
cs554-midterm-5f85d994bf-qpxmv   1/1     Running   0          26m    app=cs554-midterm,pod-template-hash=5f85d994bf
cs554-midterm-5f85d994bf-vnc72   1/1     Running   0          124m   app=cs554-midterm,pod-template-hash=5f85d994bf
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get service cs554-midterm -o wide
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE   SELECTOR
cs554-midterm   ClusterIP   10.43.18.185   <none>        80/TCP    11h   app=cs554-no-match
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get endpointslices -l kubernetes.io/service-name=cs554-midterm
NAME                ADDRESSTYPE   PORTS     ENDPOINTS   AGE
cs554-midterm-lx8mq   IPv4          <unset>   <unset>     11h
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm exec midterm-client -- wget -S -T 3 -t 1 -O - http://cs554-midterm/
--2026-10-21 05:00:00--  http://cs554-midterm/
Resolving cs554-midterm (cs554-midterm)... 10.43.18.185
Connecting to cs554-midterm (cs554-midterm)|10.43.18.185|:80... failed: Connection refused.
command terminated with exit code 4
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 12. Selector mismatch prevents Service access while application Pods remain healthy.

Reapplying `kubernetes/service.yaml` restored the original selector. Both endpoints reappeared, and a request through the Service returned HTTP 200, confirming recovery.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm apply -f kubernetes/service.yaml
service/cs554-midterm configured
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get service cs554-midterm -o wide
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE   SELECTOR
cs554-midterm   ClusterIP   10.43.18.185   <none>        80/TCP    11h   app=cs554-midterm
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get endpointslices -l kubernetes.io/service-name=cs554-midterm
NAME                ADDRESSTYPE   PORTS   ENDPOINTS                 AGE
cs554-midterm-lx8mq   IPv4          8080    10.42.0.36,10.42.0.38   11h
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm exec midterm-client -- wget -S -T 3 -t 1 -O - http://cs554-midterm/
--2026-10-21 05:02:54--  http://cs554-midterm/
Resolving cs554-midterm (cs554-midterm)... 10.43.18.185
Connecting to cs554-midterm (cs554-midterm)|10.43.18.185|:80... connected.
HTTP request sent, awaiting response...
  HTTP/1.1 200 OK
  Content-Type: application/json
  Date: Wed, 21 Oct 2026 05:02:54 GMT
  Connection: keep-alive
  Keep-Alive: timeout=5
  Transfer-Encoding: chunked
Length: unspecified [application/json]
Saving to: 'STDOUT'

     0K                                                       19.4M=0s

2026-10-21 05:02:54 (19.4 MB/s) - written to stdout [131]

{"service":"cs554-midterm","message":"Hello from my midterm service!","hostname":"cs554-midterm-5f85d994bf-qpxmv","requestCount":1}root@LAPTOP-B3C65M98:~/cs
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 13. Restoring the matching Service selector restores endpoints and HTTP access.

CS 554: Readiness Experiment

Before the experiment, both application Pods were Ready and both endpoints had `ready=true`.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# POD=$(kubectl -n cs554-midterm get pods -l app=cs554-midterm -o jsonpath='{.items[0].metadata.name}')
root@LAPTOP-B3C65M98:~/cs454-midterm-project# echo "$POD"
cs554-midterm-5f85d994bf-qpxmv
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pods -l app=cs554-midterm -o wide
NAME                              READY   STATUS    RESTARTS   AGE    IP           NODE                          NOMINATED NODE   READINESS GATES
cs554-midterm-5f85d994bf-qpxmv   1/1     Running   0          33m    10.42.0.38   k3d-cs554-midterm-server-0   <none>           <none>
cs554-midterm-5f85d994bf-vnc72   1/1     Running   0          131m   10.42.0.36   k3d-cs554-midterm-server-0   <none>           <none>
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pod "$POD" -o jsonpath='UID: {.metadata.uid}{"\n"}Container: {.status.containerSt
atuses[0].containerID}{"\n"}Restarts: {.status.containerStatuses[0].restartCount}{"\n"}'
UID: ddb2998f-d761-4363-8cd5-25e3ab231c9e
Container: containerd://69c0b5662235eb5762c50ddf7fc1e8590a7e6c994717c45d0e1f2870a44fcab5
Restarts: 0
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get endpointslices -l kubernetes.io/service-name=cs554-midterm -o jsonpath='{range .i
tems[*].endpoints[*]}{.targetRef.name}{" ready="}{.conditions.ready}{"\n"}{end}'
cs554-midterm-5f85d994bf-vnc72 ready=true
cs554-midterm-5f85d994bf-qpxmv ready=true
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 14. Readiness baseline with two ready endpoints and the selected Pod's container identity.

Creating `/tmp/midterm-not-ready` inside Pod `qpxmv` activated the application's readiness toggle, causing `/health` to return HTTP 503. The Pod remained Running but became NotReady. Its EndpointSlice entry remained listed with `ready=false`, while the other replica stayed `ready=true`.

```
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm exec "$POD" -- touch /tmp/midterm-not-ready
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm wait --for=condition=Ready=false "pod/$POD" --timeout=60s
pod/cs554-midterm-5f85d994bf-qpxmv condition met
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pods -l app=cs554-midterm -o wide
NAME                              READY   STATUS    RESTARTS   AGE    IP           NODE                          NOMINATED NODE   READINESS GATES
cs554-midterm-5f85d994bf-qpxmv   0/1     Running   0          37m    10.42.0.38   k3d-cs554-midterm-server-0   <none>           <none>
cs554-midterm-5f85d994bf-vnc72   1/1     Running   0          135m   10.42.0.36   k3d-cs554-midterm-server-0   <none>           <none>
root@LAPTOP-B3C65M98:~/cs454-midterm-project# kubectl -n cs554-midterm get pod "$POD" -o jsonpath='UID: {.metadata.uid}{"\n"}Container: {.status.containerSt
atuses[0].containerID}{"\n"}Restarts: {.status.containerStatuses[0].restartCount}{"\n"}'
UID: ddb2998f-d761-4363-8cd5-25e3ab231c9e
Container: containerd://69c0b5662235eb5762c50ddf7fc1e8590a7e6c994717c45d0e1f2870a44fcab5
Restarts: 0
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
kubectl -n cs554-midterm get endpointslices -l kubernetes.io/service-name=cs554-midterm -o jsonpath='{range .items[*].endpoints[*]}{.targetRef.name}{" ready
="}{.conditions.ready}{"\n"}{end}'
cs554-midterm-5f85d994bf-vnc72 ready=true
cs554-midterm-5f85d994bf-qpxmv ready=false
root@LAPTOP-B3C65M98:~/cs454-midterm-project#
```

Figure 15a. One replica becomes NotReady while remaining Running.

A direct root request to `/` on `qpxmv` still returned HTTP 200, confirming that the application process continued serving requests. A request through the Service returned HTTP 200 from `vnc72`, the eligible replica.

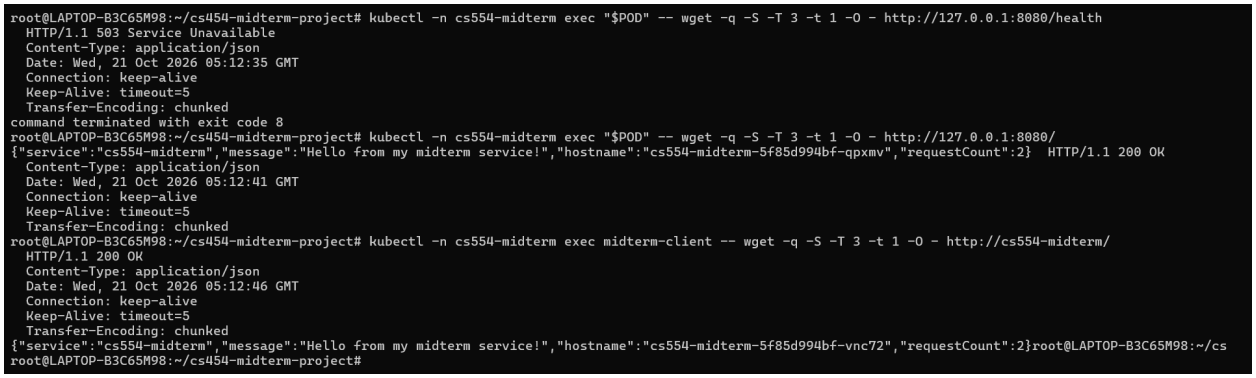


Figure 15b. The NotReady Pod continues serving root requests while the Service reaches the other ready replica.

Removing the marker restored readiness and both eligible endpoints. The Service successfully reached `qpxmv` again. This demonstrated that readiness controls Service traffic eligibility without requiring a container restart.

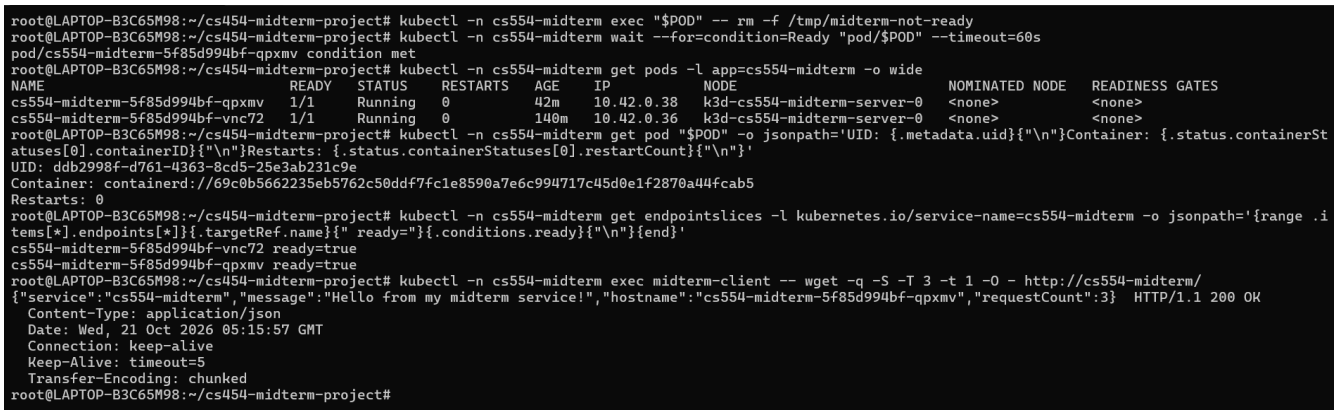


Figure 16. Removing the readiness marker restores both ready endpoints without restarting the selected container.

CS 554: Readiness Analysis

The Kubernetes resource configuration balances application needs with efficient cluster usage. Each container requests 100m CPU and 64Mi of memory, reserving a modest amount of scheduling capacity for the lightweight Node.js service. The higher limits of 500m CPU and 128Mi memory allow additional resource consumption when needed while preventing a container from consuming excessive cluster resources. CPU usage may be throttled at its limit, while exceeding the memory limit can result in termination. These settings are reasonable starting values but would require monitoring and adjustment under heavier workloads.

The readiness experiment demonstrated that application availability and process execution are separate conditions. When the readiness marker caused `/health` to return HTTP 503,

Kubernetes marked one Pod NotReady and excluded it from eligible Service endpoints. However, the application process continued running and responding to direct requests. This showed that a Pod can remain operational without being considered ready to serve traffic through Kubernetes.

The second replica remained Ready, allowing the Service to continue responding successfully despite the first replica's readiness failure. Removing the marker restored the original Pod's readiness without restarting its container, as confirmed by its unchanged identity and restart count.

Unlike the earlier Pod deletion experiment, which required Kubernetes to create a replacement, the readiness experiment demonstrated traffic management and recovery without process replacement. In all, these observations illustrate how Kubernetes uses replica management and readiness checks to maintain service availability under different failure conditions.

Part 5: OpenStack Deployment Analysis

I would use the course's ubuntu-24.04 image and inspect the m1.small flavor before launch. I would target approximately one vCPU, 2 GiB RAM, and a 20 GiB boot disk, providing modest initial capacity for Ubuntu, Docker, and this small service. The course identifies private, private-subnet, router1, and external network public. I would verify which resources already exist. [1]

The administrator would supply images, available flavors, and external connectivity. I would register my SSH public key, create an application security group, and allocate a floating IP from public. I would reuse the tenant network, subnet, and router, or create them if permitted. A separate Cinder data volume is unnecessary because the counter is temporary and the boot disk would hold the operating system and application files.

Keystone would authenticate my account and authorize access within my project. [2] Glance provides the Ubuntu VM image, which is distinct from the application's Docker image. [3] Nova provisions and schedules the instance using the selected flavor and coordinates with networking services. [4] Neutron supplies networking, addressing, routing, and connectivity controls. [5]

Once the VM exists, I would use cloud-init to install Docker Engine and Compose, verify provisioning, and build the application image from the same Dockerfile used locally. Docker Compose would then configure and start the container with docker compose up -d --wait. OpenStack provides the VM and Compose manages the application inside it.

An external client would request http://FLOATING_IP:8080/. The floating-IP mapping and Neutron routing would direct traffic to the VM's private address, subject to security-group filtering. Docker's published port would forward VM port 8080 to container port 8080 and the Node.js process. [5], [6]

I would allow inbound TCP 8080 from the intended clients. SSH would require a separate TCP 22 rule restricted to my administrative address. ICMP would be optional for ping diagnostics, since allowing ping does not enable HTTP. [6] The cloud deployment would use port 8080 directly, rather than the local VirtualBox host forward on port 18081.

If the instance were running but unreachable externally, I would investigate from the application outward. First, I would inspect `cloud-init status --long`, `docker compose ps`, and `docker compose logs` for incomplete provisioning or startup errors. I would request both endpoints inside the container, then from the VM using localhost:8080, retaining responses and errors.

If container access worked but VM access failed, I would inspect Docker's port mapping, the application's listening address, and the guest firewall. Next, I would test the VM's private address from another instance on the tenant network. Failure there would prompt checks of the

instance's network port, subnet, and security-group rules. If private access worked, I would inspect the floating-IP association, router interfaces, external gateway, and upstream reachability. I would record network configuration alongside HTTP status codes or connection errors to identify where connectivity stops.

Operating OpenStack requires maintaining identity, image storage, compute hosts, scheduling, virtual networking, and their dependencies. [4], [5] Deploying into a working cloud assumes those components already function. The course's optional DevStack guide illustrates the distinction: reaching the DevStack host does not guarantee access to instance floating IPs, because additional host routing or NAT may be required. [7] This is an infrastructure networking problem since the application could already be responding inside its VM.

For this small service, one VM with Compose would be sufficient when a single instance and simple administration meet the requirements. In my experiment, Kubernetes additionally maintained two replicas, replaced a deleted Pod, and used readiness to exclude one replica from Service traffic while its process continued running.

References

[1] UAH Computer Science, "OpenStack Class Environment," CS454 Course Exercises. Accessed: October 21, 2026.
[2] OpenStack, "Keystone, the OpenStack Identity Service." Accessed: October 21, 2026.
[3] OpenStack, "Glance Documentation: About Glance." Accessed: October 21, 2026.
[4] OpenStack, "OpenStack Compute (nova)." Accessed: October 21, 2026.
[5] OpenStack, "OpenStack Networking." Accessed: October 21, 2026.
[6] OpenStack, "Configure Access and Security for Instances." Accessed: October 21, 2026.
[7] UAH Computer Science, "Optional: Build Your Own OpenStack Lab with DevStack," CS454 Course Exercises. Accessed: October 21, 2026.